

Nordhem

A system that got them to market ten years ago, and now gets in the way of everything they want to build next. Your shared case study for the two days.

How this case study works

Everything across the two days runs on this one system. You will assess it, storm it, decide on it, design against it, and plan its migration. Keep this document with you throughout, it is the shared reference every exercise draws from.

The system is deliberately imperfect in ways that are realistic rather than cartoonish. There is no single villain module and no obviously correct first extraction. Reasonable people will disagree on where the boundaries fall and what to move first, and that disagreement is exactly where the learning lives. When you find yourself arguing with the person next to you about whether something belongs together, you are doing the work.

SECTION 1

The company

Nordhem sells Scandinavian design, homeware, lighting, furniture, and textiles, online across the Nordics and increasingly into Germany and the Netherlands. It started in 2015 as a single-country webshop in Denmark and grew country by country. Revenue is around 180 million EUR a year. It is a mid-sized business that behaves, technically, like it is still the startup it was ten years ago.



The company is profitable and growing. This matters, because it shapes every constraint that follows. There is no burning platform that justifies stopping the world to rewrite, and any modernisation has to happen while the business keeps shipping features and hitting its numbers. Leadership supports improving the architecture in principle, and loses that enthusiasm the moment a roadmap feature slips because of it.

Business context that constrains everything

The December peak

Around 40 percent of annual revenue lands in the six weeks before Christmas. Traffic runs three to four times the baseline, and a single bad hour on a December weekend can cost more than a normal day's revenue. Nobody deploys anything non-trivial from mid-November to early January. This compresses the working year and makes reliability during peak an existential concern rather than a nice-to-have.

The B2B expansion

Nordhem has signed its first business customers, interior designers and small hotel chains buying in volume, and wants to build this into a real line. B2B needs things the consumer system never had: quotes, negotiated pricing per customer, credit terms and invoicing rather than immediate card payment, approval workflows, and much larger order sizes. Right now this is being bolted onto the consumer checkout with special-case code, and it is already painful.

The mobile ambition

There is no mobile app. Leadership wants one, both for consumers and eventually a separate ordering app for B2B buyers. The current system has no clean API for a mobile client to build against; the web frontend talks to the backend through endpoints shaped entirely around the web pages that call them.

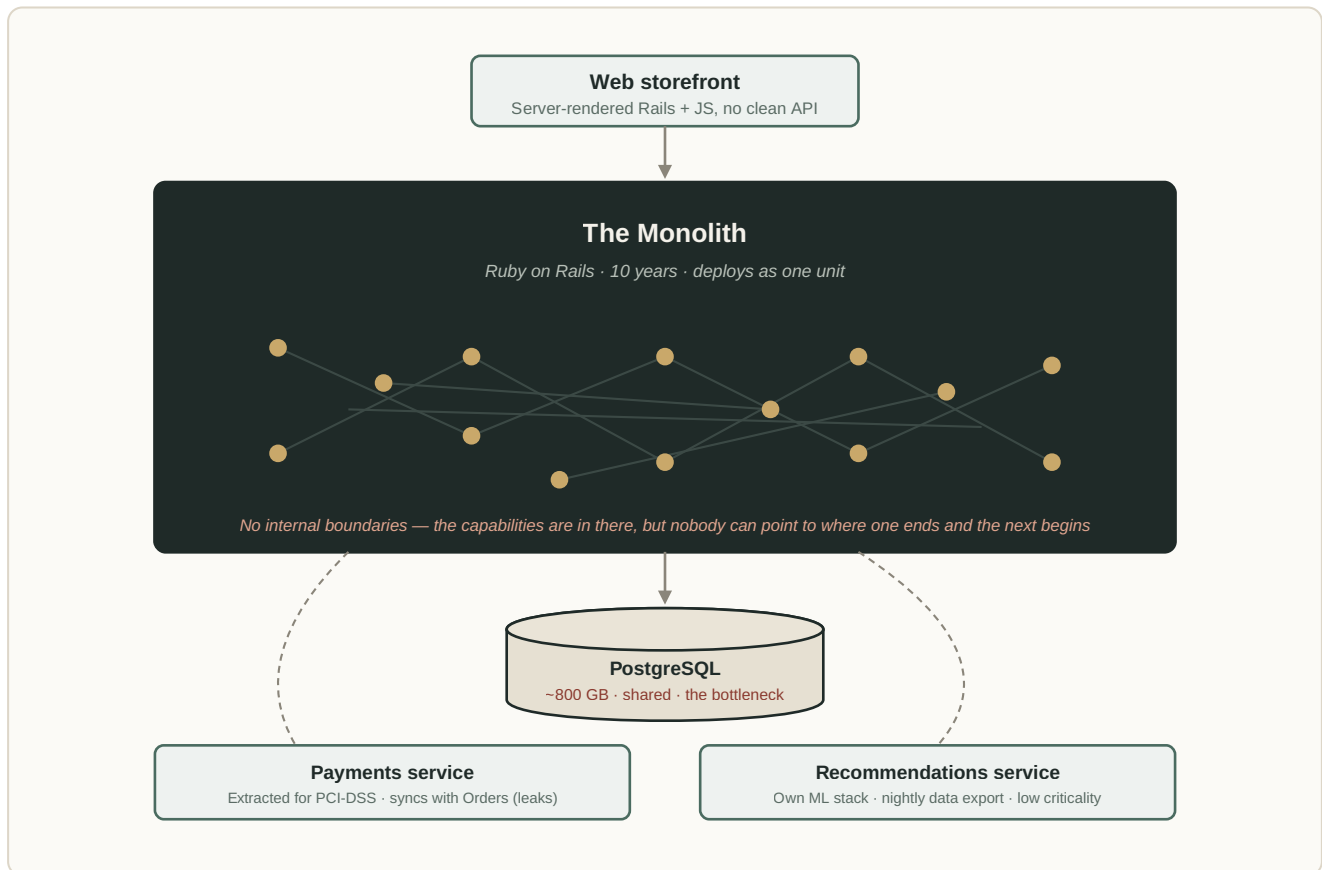
Regulation

Card payments bring PCI-DSS scope. Operating across the EU brings GDPR, which matters because customer data is currently spread across the system with no clear owner, making "delete this customer's data" a genuinely hard request to satisfy. Expanding into Germany adds country-specific invoicing and tax rules.

SECTION 2

The current architecture

One large Rails application, one large database, two services that escaped, and a web frontend welded to the whole thing.



The system today, as it actually presents: one opaque deployable unit on a shared database, with only two services carved off and a frontend welded on top. What is inside is anyone's guess until you go looking.

The monolith

The heart of the system is a Ruby on Rails application, ten years old, that has grown to cover essentially every business capability. Internally it is organised the way it was built: by technical layer and by whatever was convenient at the time. Everything the business does lives in the same codebase and ships as one unit, and the capabilities are not cleanly separated inside it, they overlap, share tables, and call into each other freely. The team calls it "the platform." Everyone knows which parts are frightening to touch, but nobody can draw you a clean map of where one capability ends and the next begins. Part of your job is to find those lines.

The database

A single large PostgreSQL database, around 800 GB, sits behind the monolith. Every part of the application reads and writes it directly, and there are no internal boundaries: the checkout code reads catalog tables, the reporting queries join across the entire schema, and a nightly batch job touches almost everything. Some of the largest and most contended tables are the ones written during checkout, which is also where the December load concentrates.

The database is the real bottleneck. Vertical scaling has been the answer three times already and there is not much headroom left. Read replicas exist, but the write path cannot be replicated away.

What has already been extracted

Two things live outside the monolith, and their histories are worth understanding because you will reason about both.

PAYMENTS · EXTRACTED ~2 YEARS AGO

Pulled out for PCI-DSS, integrating an external payment provider and keeping card handling out of scope for the rest of the system. Considered a success, and incomplete in a revealing way: the monolith still holds order and payment status in its own tables and stays in sync through a mix of synchronous calls at checkout and a reconciliation job that periodically finds discrepancies someone resolves by hand. **The boundary exists, but it leaks.**

RECOMMENDATIONS · BUILT SEPARATELY

Built by a small data/ML team, largely because they wanted their own stack and cadence rather than because of any architectural plan. Reads a copy of catalog and order data through a nightly export and serves "you might also like" results. Loosely coupled, not critical. **Extracted for team-autonomy reasons rather than domain reasons, and it shows.**

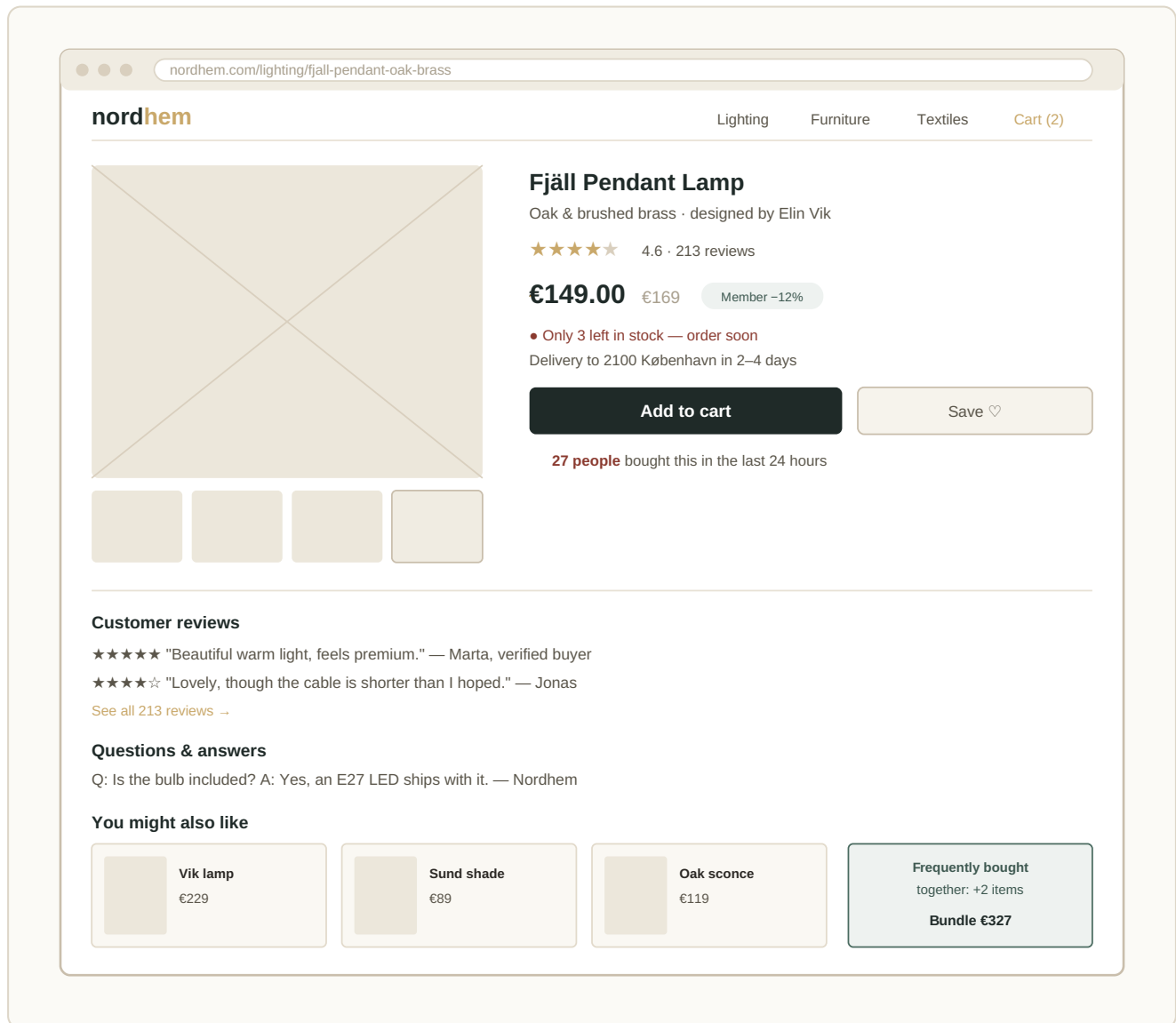
How it talks to the outside

The web storefront is a server-rendered Rails frontend with a layer of JavaScript on top, tightly coupled to the monolith that renders it. There is no clean public API. The endpoints that exist were shaped around the web pages that consume them, so they return exactly what a given page needs and nothing a different client, like a mobile app, would want.

SECTION 3

The product, on screen

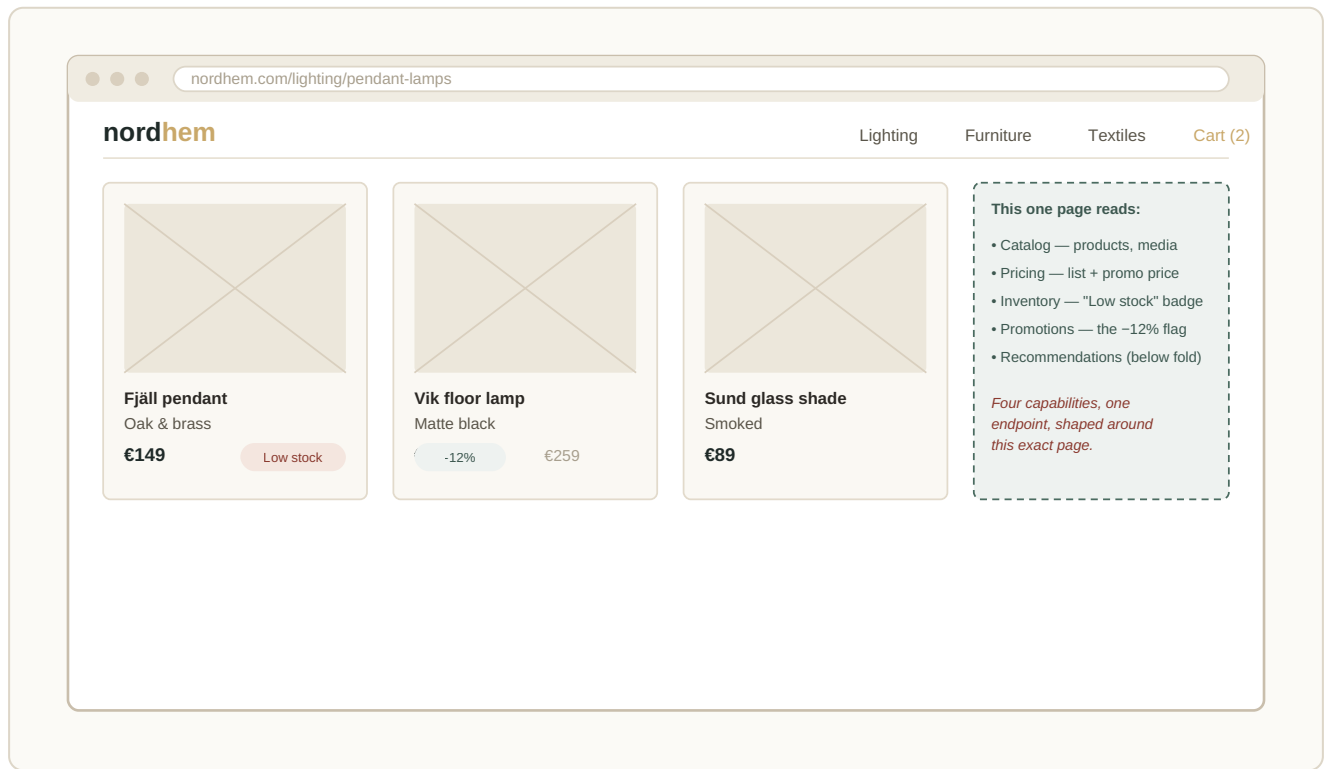
The system is not an abstraction. These are the surfaces customers actually touch, and each one quietly pulls together many different parts of the business at once. As you read these screens, ask yourself: how many distinct capabilities does this single page depend on, and where would you draw the line between them?



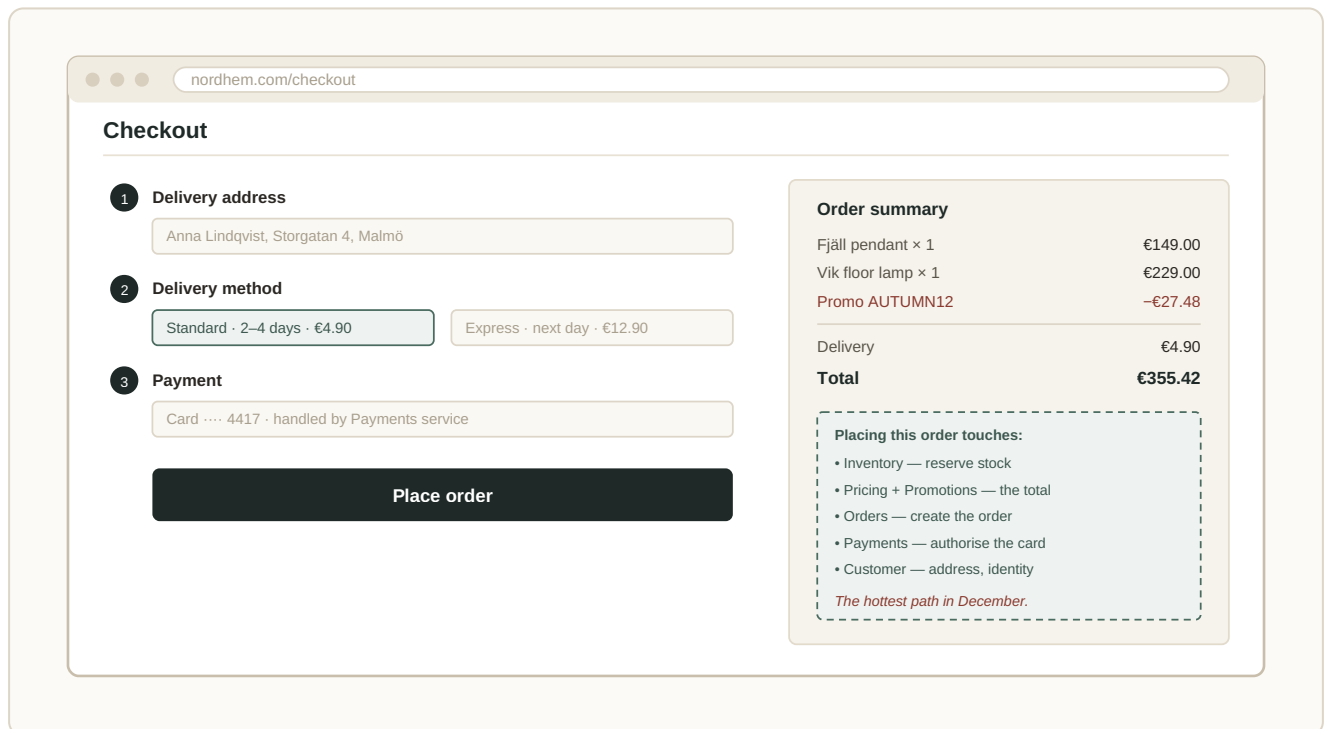
Product detail page. Count the capabilities on this single screen: product content, pricing, member and promotional pricing, live stock, delivery estimation, ratings, written reviews, customer Q&A, "bought in the last 24 hours" social proof, personalised recommendations, and a bundle offer. Where does one capability end and the next begin? Which of these belong together, and which are their own thing?

This one page is the sharpest illustration of the challenge. It looks like a single feature, "show the product," but it draws on a dozen distinct capabilities, some of which may be their own subdomains, some of which belong together, and some of which already live in separate systems. Reviews and ratings are their own world of moderation and verified-buyer logic. Q&A is another. The "27 people bought this" social proof is a different concern again, closer to analytics than to catalog. Recommendations already runs as its own

service. Pricing here is not one thing but several: a list price, a struck-through was-price, a member discount, a promotional badge, and a bundle price. A page like this is where boundary debates get real, because reasonable people will group these capabilities differently, and the way you group them shapes the architecture that follows.



Category page. A "simple" listing already fans out across four capabilities and the recommendations service.



Checkout. The convergence point: five capabilities plus the external payment provider, all on the December-peak critical path.

nordhem.com/business/quote

B2B · bolted on

Request a quote

Studio Nord AS · trade account · credit terms net-30

Line items

Vik floor lamp × 40	list €229	negotiated €189
Fjäll pendant × 60	list €149	negotiated €119

Submit for approval

None of this fits the consumer model:

- Per-customer pricing
- Quotes, not carts
- Approval workflow
- Credit & invoicing, not card capture
- Company accounts, many buyers, roles

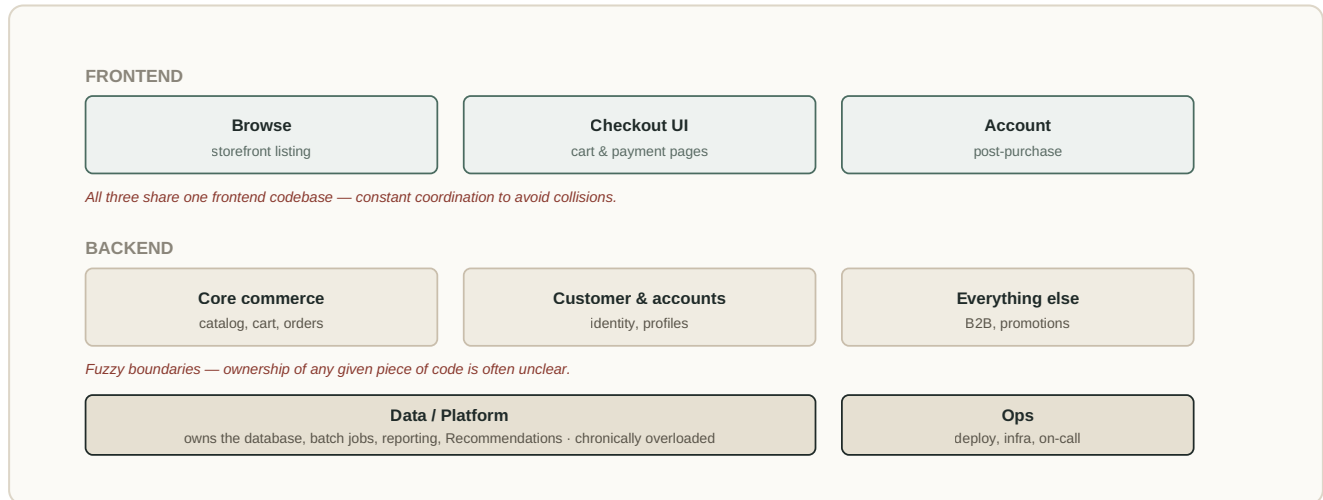
Special-cased into checkout.

B2B quote flow. Every element here is a special case grafted onto a consumer checkout that was never designed for it.

SECTION 4

The teams

Eight teams, organised by technical layer rather than business capability. This is Conway's Law in plain view, and it explains much of the pain.



Nobody owns a business capability end to end. Shipping a checkout change needs a frontend team, a backend team, and the data/platform team, none reporting to the same person below the CTO.

- **Frontend (three teams).** Split loosely by area of the storefront, browse, checkout, and account, but all working in the same frontend codebase and constantly coordinating.
- **Backend (three teams).** Split roughly by core commerce, customer and accounts, and everything else including B2B and promotions. The boundaries are fuzzy and ownership is often unclear.
- **Data/platform (one team).** Owns the database, nightly batch jobs, reporting, and Recommendations. Everyone depends on them and they are chronically overloaded.
- **Ops (one team).** Owns deployment, infrastructure, and on-call. The ones awake during the December peak.

The pain, stated plainly

The list you will recognise from your own working life. Deliberately concrete, because vague pain does not help you decide anything.

Deployments are slow and risky. Everything ships as one unit, so a release needs coordination across teams and a lot of caution. Deploys happen roughly every two weeks and are a whole-team event. A single risky change holds up everyone else's work, and during the pre-Christmas freeze nothing ships at all.

Teams block each other. Two teams changing nearby code in the same release step on each other. Merge conflicts, broken shared code, and "please don't deploy yet" are routine. Frontend teams wait on backend changes; backend teams wait on data/platform for anything touching the schema.

Features take months. A change that sounds simple, a new product attribute, a new checkout step, a new customer field, has to thread through the whole stack and multiple teams. Three to six months from idea to production is normal for anything cross-cutting.

The database is a bottleneck, technically and organisationally. Technically it cannot scale much further on the write path. Organisationally, because everything shares it, any schema change is a negotiation and the data/platform team is the constraint on everyone.

B2B is straining the seams. The consumer checkout was never designed for quotes, negotiated pricing, credit terms, or approval flows. Each B2B requirement is added as a special case, and the special cases are accumulating into something nobody wants to maintain.

Customer data has no clear home. Customer information is spread across the schema with no single owner, so GDPR requests are slow and manual, and it is hard to answer "where does this customer's data live." The B2B work will add company accounts, multiple buyers, and roles, which the current model has no concept of.

Reliability fear dominates December. Because the system is one unit sharing one database, a problem in a minor area can reach checkout. During peak, the blast radius of any single failure is the whole system, and everyone knows it.

The business capabilities

A reference map of the domain, with the genuine tensions named. Use it to orient, not to shortcut, the real work is deciding which of these become services, in what order, and where the boundaries actually fall. Nobody's answer should match this exactly.

Capability	What it covers	The tension worth arguing about
Catalog	Products, attributes, categories, media, editorial content. Stable, read-heavy.	<i>Everything reads catalog, so extracting it means confronting how many things call it.</i>
Pricing	List prices, promotions, and now B2B negotiated pricing.	<i>Is pricing part of catalog, part of promotions, or its own thing, and does B2B pricing change the answer?</i>
Inventory	Stock levels, reservations during checkout, replenishment. Highly contended at peak.	<i>Reservation couples it tightly to checkout timing, the sync-versus-async conversation in the open.</i>
Cart & Checkout	The flow that turns browsing into an order. Hottest December path.	<i>Reads catalog, pricing, inventory; writes orders; triggers payment. Where every boundary question collides.</i>
Orders	Order lifecycle, fulfilment status, history. Currently also holds payment status.	<i>What does Orders own versus merely reference? This is the Payments seam that never fully closed.</i>
Payments	Already extracted. Integrates the external provider.	<i>The boundary exists but leaks — the cautionary tale for every extraction that follows.</i>
Customer / Accounts	Identity, profiles, addresses, and soon B2B company accounts with buyers and roles.	<i>B2B fundamentally changes the model, so extracting the consumer version now risks building the wrong thing.</i>
B2B	Quotes, negotiated pricing, credit and invoicing, approval workflows.	<i>A capability of its own, or variations threaded through several existing capabilities?</i>
Promotions / Marketing	Campaigns, discount codes, storefront editorial content.	<i>Overlaps with pricing and catalog and is easy to mis-slice.</i>
Recommendations	Already extracted, loosely coupled, low criticality.	<i>Mostly a contrast case — extracted for team reasons, not domain reasons.</i>

Capability	What it covers	The tension worth arguing about
Reporting / Analytics	Cross-cutting reads over everything, served by joining across the whole database.	<i>In a decomposed world, where does cross-capability reporting get its data? Genuinely hard; you need not resolve it.</i>

WHERE THE INTERESTING ARGUMENTS HAPPEN

Most groups gravitate to **Catalog** first because it is well understood and widely depended on, and that is defensible. The richer conversations happen around **Inventory** and **Checkout**, where sync-versus-async and data ownership have no comfortable answer, and around **Customer**, where the looming B2B change means extracting the obvious thing now may mean extracting the wrong thing.